

Chapter 14. Business Logic

By now, you may be thinking, “Yes, our algorithmic ranking and recommendation has arrived! Personalization for every user with latent understanding is how we run our business.” Unfortunately, the business is rarely this simple.

Let’s take a really straightforward example, a recipe recommendation system. Consider a user who simply hates grapefruit (one of the authors of this book *really* does) but may love a set of other ingredients that go well with grapefruit: asparagus, avocado, banana, butter, cashews, champagne, chicken, coconut, crab, fish, ginger, hazelnut, honey, lemon, lime, melon, mint, olive oil, onion, orange, pecan, pineapple, raspberry, rum, salmon, seaweed, shrimp, star anise, strawberry, tarragon, tomato, vanilla, wine, and yogurt. These ingredients are the *most* popular to pair with grapefruit, and the user loves almost all of these.

What’s the right way for the recommender to handle this case? It may seem like this is something that collaborative filtering (CF), latent features, or hybrid recommendations would catch. However, if the user likes all these shared flavors, the item-based CF model would not catch this well. Similarly, if the user truly *hates* grapefruit, latent features may not be sufficient to truly avoid it.

In this case, the simple approach is a great one: *hard avoids*. In this chapter, we’ll talk about some of the intricacies of business logic intersecting the output of your recommendation system.

Instead of attempting to learn exceptions as part of the latent features that the model utilizes when making recommendations, it’s more consistent and simple to integrate these business rules as an external step via deterministic logic. As an example: the model could remove all grapefruit

cocktails that are retrieved instead of attempting to learn to rank them lower.

Hard Ranking

You can come up with a lot of examples of these phenomena when you start thinking of situations similar to our grapefruit scenario. *Hard ranking* usually refers to one of two kinds of special ranking rules:

- Explicitly removing some items from the list before ranking.
- Using a categorical feature to rank the results by category. (Note that this can even be done for multiple features to achieve a hierarchical hard ranking.)

Have you ever observed any of the following?

- A user bought a sofa. The system continues to recommend sofas to this user even though they won't need a sofa for the next five years.
- A user buys a birthday gift for a friend interested in gardening. Then the ecommerce site keeps recommending gardening tools despite the user having no interest in it.
- A parent wants to buy a toy for their child. But when the parent goes to the website where they usually buy toys, the site recommends several toys for a child a few years younger—the parent hasn't purchased from the site since the child was that age.
- A runner experiences serious knee pain and determines they can no longer go on long runs. They switch to cycling, which is lower impact. However, their local meetup recommendations are still all running oriented.

All of these cases can be relatively easy to deal with via deterministic logic. For these situations, we would prefer *not* to try to learn these rules via ML. We should assume that for these types of scenarios, we will get low signal about these preferences: negative implicit feedback is often lower in relevance, and many of the situations listed are represented by details that you want the system to learn once and for all. Additionally, in some of the previous examples, it can be upsetting or harmful to a relationship with a user to have the preferences not respected.

The name for these preferences is *avoids*—or sometimes constraints, overrides, or hard rules. You should think of them as explicit expectations of the system: “Don’t show me recipes with grapefruit,” “No more sofas,” “I don’t like gardening,” “My child is older than 10 now,” and “Don’t show me trail runs.”

Learned Avoids

Not all business rules are such obvious avoids that derive from explicit user feedback, and some derive from explicit feedback not directly related to specific items. It’s important to include a wide variety of avoids when considering serving recommendations.

For the sake of simplicity, let’s assume you’re building a fashion recommender system. Examples of more subtle avoids include the following:

Already owned items

These are items that users really need to purchase only once—for example, clothing users have bought through your platform or told you they already own. Creating a *virtual closet* might be a way to ask users to tell you what they have, to assist in these avoids.

Disliked features

These are features of items that the user can indicate disinterest in. During an onboarding questionnaire, you may ask users if they like polka dots or if they have a favorite color palette. These are explicitly indicated pieces of feedback that can be used for avoids.

Ignored categories

This is a category or group of items that doesn’t resonate with the user. This can be implicit but learned outside the primary recommender model. Maybe the user has never clicked the Dresses category on your ecommerce website because they don’t enjoy wearing them.

Low-quality items

Over time, you'll learn that some items are simply low quality for most users. You can detect this via a high number of returns or low ratings from buyers. These items ultimately should be removed from inventory, but in the meantime, it's important to include them as avoids for all but the strongest signal of match.

These additional avoids can be implemented easily during the serving stage and can even include simple models. Training linear models to capture some of these rules and then applying them during serving can be a useful and reliable mechanism for improving ranking. Note that the small models perform very fast inference, so little negative impact usually results from including them in the pipeline. For larger-scale behavior trends or higher-order factors, we expect our core recommendation models to learn these ideas.

Hand-Tuned Weights

On the other side of the spectrum of avoids is *hand-tuned ranking*. This technique was popular in earlier days of search ranking, when humans would use analytics and observation to determine what they thought were the most important features in a ranking and then craft a multiobjective ranker. For example, flower stores may rank higher in early May as many users search for Mother's Day gifts. Since there could be many variable elements to track, these kinds of approaches don't scale well and have been largely deemphasized in modern recommendation ranking.

However, hand-tuned ranking can be incredibly useful as an *avoid*. While technically it's not an avoid, we sometimes still call it that. An example of this in practice is to know that new users like to start with a lower-priced item while they're learning whether your shipping is trustworthy. A useful technique is to then uprank lower-priced items before the first order.

While it may feel bad to consider building a hand-tuned ranking, it's important to not count this technique out. It has a place and is often a great place to start. One interesting human-in-the-loop application of this kind of technique is for hand-tuned ranking by experts. Back to our fashion recommender, a style expert may know that this summer's trending color

is mauve, especially among the younger generation. Then can positively influence user satisfaction if the expert ranks these mauve items up for users in the right age persona.

Inventory Health

A unique and somewhat contentious side of hard ranking is inventory health. Notoriously hard to define, *inventory health* estimates how good the existing inventory is for satisfying user demand.

Let's take a quick look at one way to define inventory health, via affinity scores and forecasting. We can do this by leveraging a demand forecast, which is an incredibly powerful and popular way to optimize the business: what are the expected sales in each category over the next N time periods? Building these forecasting models is outside the scope of this book, but the core ideas are well captured in the famous book ["Forecasting: Principles and Practice"](#) by Rob Hyndman and George Athanasopoulos (Otexts). For the sake of our discussion, assume that you're able to roughly approximate the number of socks you'll sell over the next month, broken down by size and usage type. This can be a really instructive estimate for the number of socks of various types you should have on hand.

However, it doesn't stop there; inventory may be finite, and in practice inventory is often a major constraint on businesses that sell physical goods. With that caveat, we have to turn to the other side of the market demand. If our demand outstrips our availability, we are ultimately disappointing users who don't have access to the item they desired.

Let's take an example of selling bagels; you've calculated average demand for poppy seed, onion, asiago cheese, and egg. On any given day, many customers will come to buy a bagel with a clear preference in mind, but will you have enough of that bagel? Every bagel you don't sell is wasted; people like fresh bagels. This means that the bagels you recommend to each person are dependent on good inventory. Some users are less picky; they can get one of two or three of the options and be just as happy. In that case, it's better to give them another bagel option and save the lowest

inventory for the picky ones. This is a kind of model refinement called *optimization*, which has a huge number of techniques. We won't get into optimization techniques, but books on mathematical optimization or operations research will provide direction. *Algorithms for Optimization* by Mykel J. Kochenderfer and Tim A. Wheeler (MIT Press) is a good place to start.

Inventory health ties back to hard ranking, because actively managing inventory as part of your recommendations is an incredibly important and powerful tool. Ultimately, inventory optimization will degrade the perceived performance of your recommendations, but by including it as part of your business rules, the overall health of your business and recommender system improves. This is why it is sometimes called *global optimization*.

The reason that these methods stir up heated discussions is that not everyone agrees that the quality of recommendations for some users should be depressed to improve those for the “greater good.” Health of the marketplace and average satisfaction are useful metrics to consider, but ensure that these are aligned with the north-star metrics for the recommendation system at large.

Implementing Avoids

The simplest approach to handling avoids is via downstream filtering. To do this, you'll want to apply the avoid rules for the user before the recommendations are passed along from the ranker to the user. Implementing this approach looks something like this:

```
import pandas as pd

def filter_dataframe(df: pd.DataFrame, filter_dict: dict):
    """
    Filter a dataframe to exclude rows where columns have certain values.

    Args:
        df (pd.DataFrame): Input dataframe.
        filter_dict (dict): Dictionary where keys are column names
```

and values are the values to exclude.

Returns:

pd.DataFrame: Filtered dataframe.

"""

```
for col, val in filter_dict.items():
```

```
    df = df.loc[df[col] != val]
```

```
return df
```

```
filter_dict = {'column1': 'value1', 'column2': 'value2', 'column3': 'value3'}
```

```
df = df.pipe(filter_dataframe, filter_dict)
```

Admittedly, this is a trivial but also relatively naive attempt at avoids.

First, working purely in pandas will limit some of the scalability of your recommender, so let's convert this to JAX:

```
import jax
```

```
import jax.numpy as jnp
```

```
def filter_jax_array(arr: jnp.array, col_indices: list, values: list):
```

```
    """
```

Filter a jax array to exclude rows where certain columns have certain values

Args:

arr (jnp.array): Input array.

col_indices (list): List of column indices to filter on.

values (list): List of corresponding values to exclude.

Returns:

jnp.array: Filtered array.

```
    """
```

```
assert len(col_indices) == len(values),
```

```
masks = [arr[:, col] != val for col, val in zip(col_indices, values)]
```

```
total_mask = jnp.logical_and(*masks)
```

```
return arr[total_mask]
```

But there are deeper issues. The next issue you may face is where that collection of avoids is stored. An obvious place is somewhere like a NoSQL database keyed on users, and then you can get all of the avoids as

a simple lookup. This is a natural use of feature stores, as you saw in [“Feature Stores”](#). Some avoids may be applied in real time, while others are learned upon user onboarding. Feature stores are a great place to house avoids.

The next potential gotcha with our naive filter is that it doesn’t naturally extend to covariate avoids, or more complicated avoid scenarios. Some avoids are actually dependent on context—a user who doesn’t wear white after Labor Day, users who don’t eat meat on Fridays, or coffee-processing methods that don’t mesh well with certain brewers. All of these require conditional logic. You might think that your powerful and effective recommendation system model can certainly learn these details, but this is true only sometimes. The reality is that many of these kinds of considerations are lower signal than the large-scale concepts your recommendation system should be learning, and thus are hard to learn consistently. Additionally, these kinds of rules are often ones you should require, as opposed to remain optimistic about. For that reason, you often should explicitly specify such restrictions.

This specification can often be achieved by explicit deterministic algorithms that impose these requirements. For the coffee problem, one of the authors hand-built a decision stump to handle a few bad combinations of coffee roast features and brewers—*anaerobic espresso?! Yuck!*

Our other two examples (not wearing white after Labor Day and not eating meat on Fridays), however, are a bit more nuanced. An explicit algorithmic approach may be tricky to handle. How do we know that a user doesn’t eat meat on Fridays during one period of the year?

For these use cases, model-based avoids can impose these requirements.

Model-Based Avoids

In our quest to include more complicated rules and potentially learn them, we may sound like we’re back in the realm of retrieval. Unfortunately, even with models like wide-and-deep with lots of parame-

ters doing both user modeling and item modeling, learning such high-level relationships can be tricky.

While most of this book has focused on working fairly large and deep, this part of recommendation systems is well suited for simple models. For feature-based binary predictions (should this be recommended), we certainly have a zoo of good options. The best approach would obviously depend heavily on the number of features involved in implementing the avoid you wish to capture. It's useful to remember that many avoids that we're considering in this section start out as assumptions or hypotheses: we think some users may not wear white after Labor Day, and then attempt to find features that model this outcome well. In this way, it can be more tractable using extremely simple regression models to find covarying features with the outcome in question.

Another related piece of this puzzle is latent representations. For our Friday vegetarians, we may be trying to infer a particular persona that we know has this rule. That persona is a latent feature that we hope to map from other attributes. It's important to be careful with this kind of modeling (in general, personas can be a bit nuanced and worthy of thoughtful decision making), but it can be quite helpful. It may seem like the user-modeling parts of your large recommender model should learn these—and they can! A useful trick is to pull forward personas learned from that model and regress them against hypothesized avoids to allow for more signal. However, the other model doesn't always learn these personas because our loss functions for retrieval relevance (and downstream for ranking) are attempting to parse out relevance for individual users from the latent persona features—which may predict these avoids only amid context features.

All in all, implementing the avoids is both very easy and very hard. When building production recommendation systems, the journey is not over when you get to serving; many models factor into the final step of the process.

Summary

Sometimes you need to rely on more classic approaches to ensuring that the recommendations you're sending downstream are satisfying essential rules of your business. Learning explicit or subtle lessons from your users can be turned into simple strategies to continue to delight them.

However, this is not the end of our serving challenge. Another kind of downstream consideration is related to the kind of filtering we've done here but derives from user preference and human behavior. Ensuring that recommendations are not repeated, rote, and redundant is the subject of the next chapter on diversity in recommendations. We will also discuss how to balance multiple priorities simultaneously when determining exactly what to serve.